CS521 HW1

sli602 Siyu Li

September 2026

All the codes are provided in github repo: https://github.com/SiyuLi915/CS521

1 Question 1

1.

```
adv_x = x - eps * x.grad.sign()
```

Results:
Original Class: 2
New Class: 0
tensor(0.5000)
2.
The attempt is unsuccessful. I changed `t = 0` to `t = 1` and reran the same one-liner from the last part, and the network still says class 2, so `assert(new_class == t)` fails. My first thought was that ϵ was just too small, so I tried bigger values too, but it never lands on class 1:

ϵ	logits (class 0, 1, 2)	predicted class
0 (original x)	$(-0.0529,\ -0.0323,\ \mathbf{0.0374})$	2
0.5	$(-0.0147,\ 0.0115,\ \mathbf{0.0606})$	2
1.0	$(0.0208,\ 0.0315,\ \mathbf{0.0386})$	2
1.5	$(\mathbf{0.0524},\ 0.0457,\ 0.0308)$	0

FGSM is basically a linear approximation of the network around x. It uses the gradient at x to guess which direction is best, then takes one big step. The step actually *does* help class 1 (its logit goes from -0.032 to $+0.012$), but it pushes class 2 up even more ($0.037 \to 0.061$). The issue is that N is made of ReLUs, and a step of size 0.5 is big enough to switch a lot of them on or off. I checked, and 5 of the 10 ReLUs in each hidden layer change state. So at x' the network is behaving like a pretty different function than the one the gradient at x described. The gradient is only trustworthy close to x, and 0.5 isn't close. If you keep increasing ϵ, class 0 takes over before class 1 ever gets on top.
I'd say it means FGSM is a weak attack here, not that the network is robust.

Then I tried brute force methods. FGSM's output is always a corner $x + \epsilon s$ of the L_∞ ball, with each entry of s equal to ± 1. In 10 dimensions there are only $2^{10} = 1024$ corners, so the code just classifies all of them in one batch and keeps a corner that gets class 1.

```
import itertools
```

```
# all 2^10 = 1024 sign patterns s in {-1,+1}^10
signs = torch.tensor(list(itertools.product([-1., 1.], repeat=10)))
cands = x.detach() + eps * signs
preds = N(cands).argmax(dim=1)
adv_x = cands[preds == t][:1]
```

Every candidate is automatically within ϵ of x, since each coordinate moves by exactly ϵ. It works, and both asserts pass:

```
New Class:  1
tensor(0.5000)
```

	logits (class 0, 1, 2)
x	$(-0.0529,\ -0.0323,\ \mathbf{0.0374})$
x'	$(0.0103,\ \mathbf{0.0176},\ 0.0137)$

The distance between them is

$$\|x - x'\|_\infty = 0.5000, \qquad \|x - x'\|_2 = 0.5\sqrt{10} \approx 1.581.$$

2 Question 2

2.1 Single-norm robust accuracy

We evaluate three pre-trained PreAct ResNet-18 models on the full CIFAR-10 test set (10,000 images) using untargeted PGD with $k = 10$ steps and step size $\epsilon/4$, with $\epsilon_\infty = 8/255$ and $\epsilon_2 = 0.75$.

Model	**Clean**	ℓ_∞ **PGD** ($\epsilon = 8/255$)	ℓ_2 **PGD** ($\epsilon = 0.75$)
ℓ_∞	82.80	**51.21**	48.82
ℓ_2	**88.75**	30.85	54.39
RAMP	81.19	49.75	**59.67**

Table 1: Standard (clean) and robust accuracy (%) of the three pre-trained models on the CIFAR-10 test set. Best result in each column in bold.

The ℓ_∞ model gets the best ℓ_∞ robust accuracy (51.21%), and the ℓ_2 model has the highest clean accuracy (88.75%). That makes sense, since ℓ_2 training is usually a gentler constraint than ℓ_∞ at 8/255. But the specialists pay for it on the other norm. The ℓ_2 model is the clearest example: it drops to 30.85% under ℓ_∞, more than 23 points below its ℓ_2 number. So being robust to one norm definitely doesn't give you robustness to the other for free.

RAMP actually beats the ℓ_2 specialist on ℓ_2 (59.67% vs. 54.39%). I expected ℓ_2 to win its own column. My best guess is that the ℓ_2 model was trained with $\epsilon = 0.5$ (the usual choice for CIFAR-10), so testing it at $\epsilon = 0.75$ pushes it outside the radius it was trained for. RAMP trains against several norms at once, so it seems to hold up better at the bigger radius.

2.2 Multi-norm (union) robust accuracy

For the union threat model $\Delta = B_2(x, \epsilon_2) \cup B_\infty(x, \epsilon_\infty)$, I ran both attacks on every test image. A point only counts for the union if it survives *both* attacks, so the union accuracy is just the ℓ_∞ accuracy minus the points that survive ℓ_∞ but get broken by ℓ_2. I counted these to see which attack is actually doing the damage (Table 2).

Model	**Union** ($\ell_\infty \cup \ell_2$)	**Survive** ℓ_∞**, broken by** ℓ_2
ℓ_∞	47.04	417
ℓ_2	30.85	0
RAMP	**49.74**	1

Table 2: Union robust accuracy (%) and the number of CIFAR-10 test images (out of 10,000) that survive the ℓ_∞ attack but get broken by the ℓ_2 attack. Best union accuracy in bold.

For ℓ_2, not a single image that survives ℓ_∞ gets broken by ℓ_2, so its union is exactly its ℓ_∞ accuracy (30.85%). RAMP is almost the same, with only 1 image. So for these two models the ℓ_2 attack adds basically nothing, and ℓ_∞ is the whole story. The ℓ_∞ model is the only one where ℓ_2 really matters: 417 images survive ℓ_∞ but get broken by ℓ_2, which is why its union (47.04%) ends up below both of its single-norm numbers.

RAMP has the best union accuracy at 49.74%, which is 2.7 points higher than ℓ_∞. It gets there by keeping its ℓ_∞ robustness close to the ℓ_∞ specialist (49.75% vs. 51.21%) while losing almost nothing extra to ℓ_2. The cost is clean accuracy: it's 1.61 points below ℓ_∞ and 7.56 points below ℓ_2.

3 Question 3

Summary

This paper points out that most adversarial robustness work trains and tests against the same attack (usually an ℓ_∞ ball), which isn't realistic, since real attackers can use attacks the defender never saw. To measure robustness to these "unforeseen" attacks, the authors build 19 differentiable non-ℓ_p attacks, like Snow, Wood, Glitch, and JPEG. Each one is controlled by some latent variables with an ℓ_p bound, so it can be run with regular PGD. Eight of these attacks make up the ImageNet-UA and CIFAR-10-UA benchmarks, and the score, UA2, is the average accuracy across those eight attacks. The paper uses this standardized benchmark to evaluate classical L_p training techniques, showing that the common practice of L_∞ training and evaluation may be misleading, as L_2 shows higher unforeseen robustness.

Strengths

1. **Important question.** Testing on attacks you didn't train on is much closer to the real world than the standard setup, where the defense already knows the attack.

2. **Simple, reusable attack design.** Every attack has the same form: you optimize a bounded set of latent variables with PGD, and ϵ controls the strength. This makes the attacks easy to use on other models and datasets, and the code is released.

3. **Large evaluation with interesting results.** They test a lot of models. Some of the results are surprising: for example, ConvNeXt-V2-large gets 0% accuracy under ℓ_∞ PGD but still 19.2% UA2, which shows the two kinds of robustness really are different.

Weaknesses

1. **The attacks might not be strong enough.** Everything uses plain PGD. The paper itself shows that AutoAttack drops DINOv2's ℓ_∞ accuracy from 15.3% to 0%, so DINOv2's top UA2 score might be inflated in the same way. *Fix:* Use stronger attacks, like adaptive step sizes and multiple restarts, at least on the best models.

2. **The metric is an average.** UA2 averages accuracy over the eight attacks, but a real attacker would just pick whichever attack works best on each image. Also, the strength of each attack is picked by hand, so some attacks are much harder than others. *Fix:* Also report worst-case accuracy over all attacks, and use the human study to set each attack's strength so they're equally hard for people.

3. **Some claims don't match the tables.** For example, the main text says the MNG method improves on the "best ℓ_p baseline" from 21.4% to 51.1%, but the stronger ℓ_∞ baseline in Table 13 is actually 39.8%. A few other numbers in the text don't match the tables either. *Fix:* A careful proofreading pass to make the text and tables consistent.

4. **The writing has a lot of typos and grammar mistakes.** For example, "make us of" (Section 2), "optimse within the peturbation set" (Section 3), and "for empirical an empirical comparison" (Section 2). These don't change the results, but they make the paper harder to read and make it feel a bit rushed. *Fix:* A careful proofreading pass, or running it through a grammar checker, before the final version.

Extensions

I believe the same idea works really well for LLM jailbreaks. Right now, jailbreak defenses are usually tested against the same kinds of attacks they were trained on, such as GCG adversarial suffixes, but people keep inventing new ones: role-play prompts, ciphers, switching to less common languages, and so on. Following this paper, an extension would include:

- treat each type of jailbreak as a different "attack," with a budget like the number of adversarial tokens;
- train a defense on some types and test it on held-out ones;
- report the average (and worst-case) refusal rate on the held-out types;
- check that the attacked prompt still asks for the same thing, similar to the paper's human study.

Also, I think it can also be used for Vision-language models. Models like DINOv2 and CLIP are now the image encoders inside multimodal chatbots. We could apply attacks like Snow or Wood to the image input of a VLM and check whether it misdescribes the image or can be pushed past its safety rules.

Conclusion

The main idea and the attack library are useful, but stronger attacks and a cleaner, more consistent write-up are needed for examination before fully trusting the conclusions.